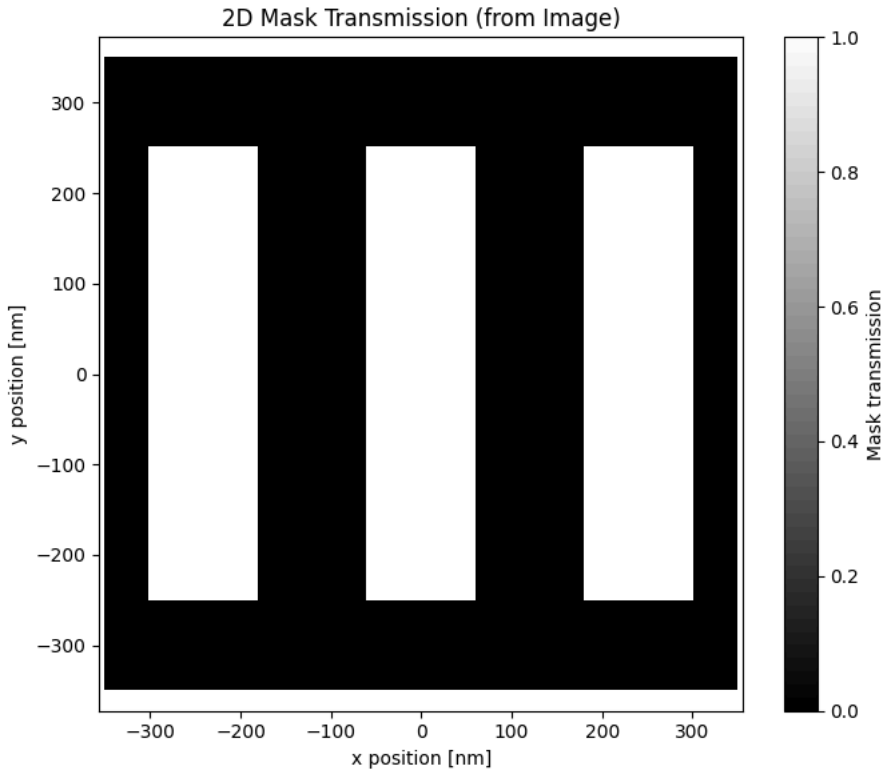
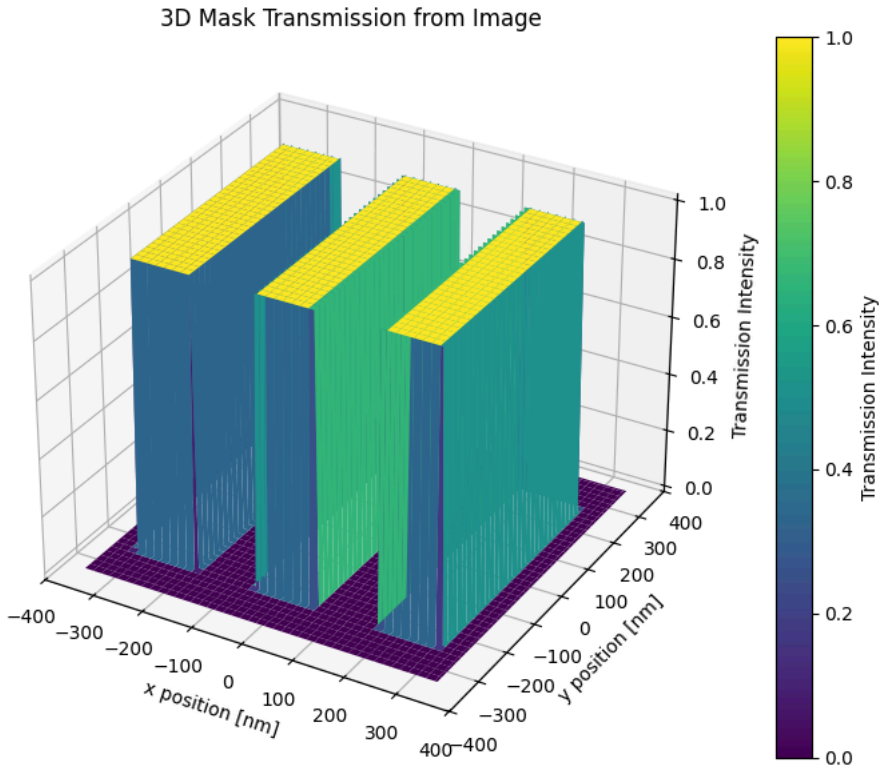
```
import numpy as np
import matplotlib.pyplot as plt
import cv2
from scipy.ndimage import gaussian_filter
from mpl_toolkits.mplot3d import Axes3D
```

```
# === Simulation Parameters ===
nx, ny = 128, 128  # Target resolution
nm_per_pixel = 1.0  # Physical size of each original image pixel in nm

# === Load and process mask image ===
image_path = '/content/ILT_contact_120.png'  # Path to input image
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if img is None:
    print(f"Error: Could not load image from path: {image_path}")
else:
    h, w = img.shape

    # Compute physical field of view (FoV) of the original image
    fov_x = w * nm_per_pixel
    fov_y = h * nm_per_pixel
    fov = max(fov_x, fov_y)

    # Pad image to make it square
    size = max(h, w)
    pad_h = (size - h) // 2
    pad_w = (size - w) // 2
    padded_img = np.pad(img, ((pad_h, size - h - pad_h), (pad_w, size - w - pad_w)), mode='constant')

    # Resize padded image to simulation resolution
    img_resized = cv2.resize(padded_img, (nx, nx), interpolation=cv2.INTER_AREA)

    # Convert to binary mask: 1 = transparent (glass), 0 = opaque (chromium)
    threshold = nx
    mask = (img_resized > threshold).astype(float)

    # === Update pixel size and coordinate grid ===
    dx = fov / nx
    dy = fov / ny
    x = np.linspace(-fov / 2, fov / 2, nx)
    y = np.linspace(-fov / 2, fov / 2, ny)
    X, Y = np.meshgrid(x, y)

    # Define length_x and length_y for plotting extent
    length_x = fov
    length_y = fov
```

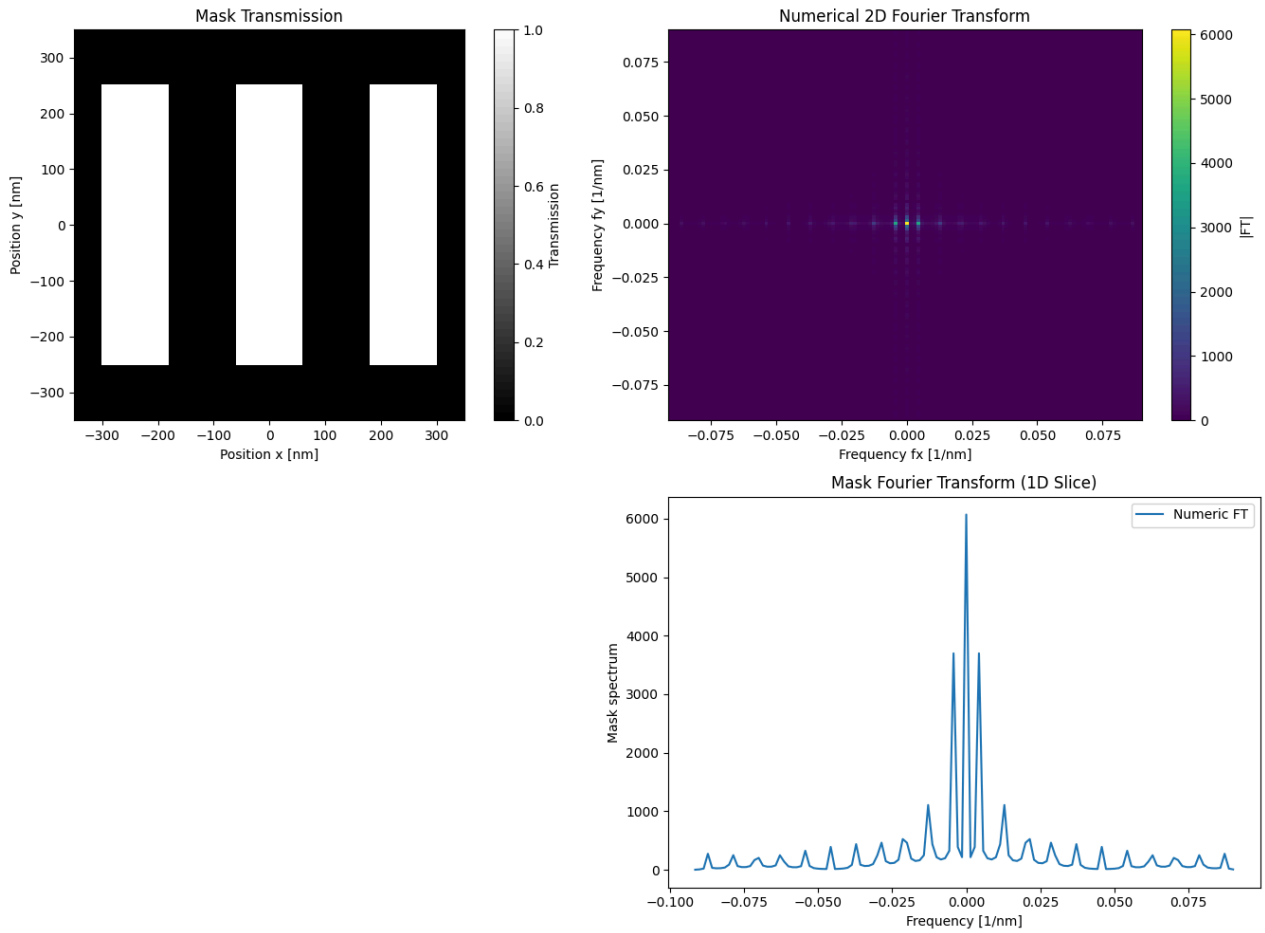
```
# === Plot the mask ===
plt.figure(figsize=(7, 6))
plt.imshow(mask, extent=[x.min(), x.max(), y.min(), y.max()],
           origin='lower', cmap='gray')
plt.title('2D Mask Transmission (from Image)')
plt.xlabel('x position [nm]')
plt.ylabel('y position [nm]')
plt.colorbar(label='Mask transmission')
plt.axis('equal')
plt.tight_layout()
plt.show()
```




```
## TRIAL ##

# === Plotting the 3D surface ===
fig = plt.figure(figsize=(10, 6))
ax = fig.add_subplot(111, projection='3d')

# Plot the transmission mask
surf = ax.plot_surface(X, Y, mask, cmap='viridis', edgecolor='none')

# Labels and title
ax.set_title('3D Mask Transmission from Image')
ax.set_xlabel('x position [nm]')
ax.set_ylabel('y position [nm]')
ax.set_zlabel('Transmission Intensity')

# Colorbar
fig.colorbar(surf, ax=ax, label='Transmission Intensity')

plt.tight_layout()
plt.show()
```

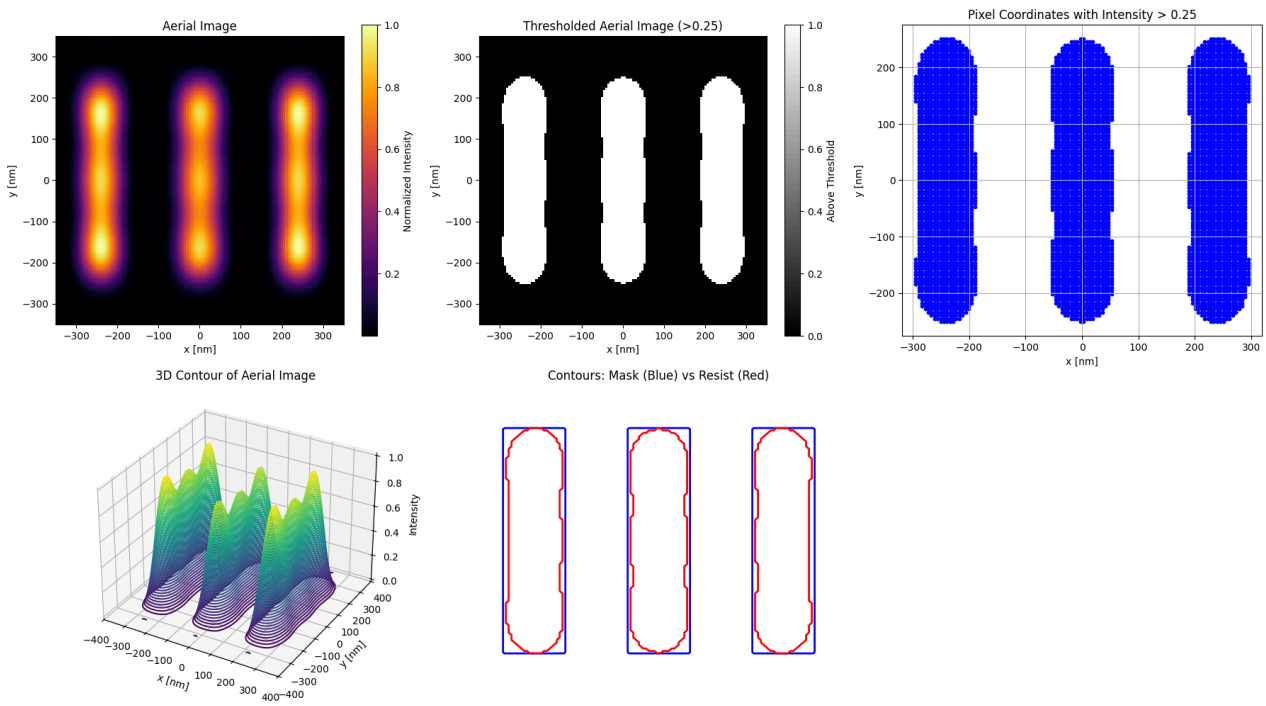



```
# Computation of the mask TF and its frequency support
mask_ft = np.fft.fftshift(np.fft.fft2(mask))

# Create frequency support grids
fx = np.fft.fftshift(np.fft.fftfreq(nx, d=dx))
fy = np.fft.fftshift(np.fft.fftfreq(ny, d=dy))
FX, FY = np.meshgrid(fx, fy)

# Analytical Fourier Transform (2D sinc function)
# Using fov_x and fov_y as feature dimensions for approximation
mask_ana_ft = fov_x * fov_y / (dx * dy) * np.sinc(fov_x * FX) * np.sinc(fov_y * FY)


# Create plots
plt.figure(figsize=(14, 10))

# Plot the mask
plt.subplot(2, 2, 1)
plt.imshow(mask, extent=[-length_x/2, length_x/2, -length_y/2, length_y/2],
           cmap='gray', origin='lower')
plt.title('Mask Transmission')
plt.xlabel('Position x [nm]')
plt.ylabel('Position y [nm]')
plt.colorbar(label='Transmission')

# Plot the magnitude of the numerical FT
plt.subplot(2, 2, 2)
plt.imshow(np.abs(mask_ft), extent=[fx.min(), fx.max(), fy.min(), fy.max()],
          cmap='viridis', origin='lower', aspect='auto')
plt.title('Numerical 2D Fourier Transform')
plt.xlabel('Frequency fx [1/nm]')
plt.ylabel('Frequency fy [1/nm]')
plt.colorbar(label='|FT|')


# Plot 1D slice for comparison with original plot
plt.subplot(2, 2, 4)
center_y = nx // 2
plt.plot(fx, np.abs(mask_ft[center_y, :]), label='Numeric FT')
plt.title('Mask Fourier Transform (1D Slice)')
plt.xlabel('Frequency [1/nm]')
plt.ylabel('Mask spectrum')
plt.legend()

plt.tight_layout()
plt.show()
```




```
# Fourier Transform of mask
mask_ft = np.fft.fftshift(np.fft.fft2(mask))

# Frequency coordinates
fx = np.fft.fftshift(np.fft.fftfreq(nx, d=dx))
fy = np.fft.fftshift(np.fft.fftfreq(ny, d=dy))
FX, FY = np.meshgrid(fx, fy)

# Pupil filter
NA = 1.35
wavelength = 193.  # nm
cutoff_freq = NA / wavelength
pupil_filter = (np.sqrt(FX**2 + FY**2) <= cutoff_freq).astype(float)

# Filter and inverse FT
filtered_ft = mask_ft * pupil_filter
aerial_image = np.abs(np.fft.ifft2(np.fft.ifftshift(filtered_ft))) ** 2
aerial_image /= np.max(aerial_image)

# Thresholding
threshold = 0.25
binary_image = aerial_image > threshold
x_above, y_above = X[binary_image], Y[binary_image]

# === Plotting ===
fig = plt.figure(figsize=(18, 10))

# Aerial Image
ax1 = fig.add_subplot(2, 3, 1)
im1 = ax1.imshow(aerial_image, extent=[x.min(), x.max(), y.min(), y.max()],
                 cmap='inferno', origin='lower')
ax1.set_title('Aerial Image')
```

```
ax1.set_xlabel('x [nm]')
ax1.set_ylabel('y [nm]')
fig.colorbar(im1, ax=ax1, label='Normalized Intensity')

# Thresholded Aerial Image
ax2 = fig.add_subplot(2, 3, 2)
im2 = ax2.imshow(binary_image, extent=[x.min(), x.max(), y.min(), y.max()],
                 cmap='gray', origin='lower')
ax2.set_title(f'Thresholded Aerial Image (>{threshold})')
ax2.set_xlabel('x [nm]')
ax2.set_ylabel('y [nm]')
fig.colorbar(im2, ax=ax2, label='Above Threshold')

# Exposed Pixels (Scatter)
ax3 = fig.add_subplot(2, 3, 3)
ax3.scatter(x_above, y_above, s=10, color='blue')
ax3.set_title(f'Pixel Coordinates with Intensity > {threshold}')
ax3.set_xlabel('x [nm]')
ax3.set_ylabel('y [nm]')
ax3.grid(True)
ax3.axis('equal')

# === 3D Contour Plot of Aerial Image ===
ax4 = fig.add_subplot(2, 3, 4, projection='3d')
contours = ax4.contour3D(X, Y, aerial_image, levels=50, cmap='viridis')
ax4.set_title("3D Contour of Aerial Image")
ax4.set_xlabel("x [nm]")
ax4.set_ylabel("y [nm]")
ax4.set_zlabel("Intensity")

# === Contour plot ===
ax5 = fig.add_subplot(2, 3, 5)
ax5.contour(mask, levels=[0.5], colors='blue', linewidths=2)
ax5.contour(binary_image, levels=[0.5], colors='red', linewidths=2)
ax5.set_title("Contours: Mask (Blue) vs Resist (Red)")
ax5.axis('off')


plt.tight_layout()
plt.show()
```




```
## TRIAL ##

# # === 1. Forward Lithography Model (Gaussian PSF) ===
# def forward_model(mask, sigma=2.0):
```

```
# def forward_model(mask, sigma=2.0):
#     return gaussian_filter(mask, sigma=sigma)

# # === 2. Resist Model (Sigmoid) ===
# def resist_model(image, a=10, t=0.5):
#     return 1 / (1 + np.exp(-a * (image - t)))

# # === 3. Cost Function ===
# def compute_cost(predicted, target):
#     return np.mean((predicted - target) ** 2)

# # === 4. ILT Gradient Descent Optimizer ===
# def ilt_gradient_descent(target, sigma=2.0, a=10, t=0.5,
#                          lr=0.1, beta=0.01, iterations=200):
#     mask = np.random.rand(*target.shape)  # Random initial mask
#     cost_history = []

#     for i in range(iterations):
#         aerial = forward_model(mask, sigma)
#         resist = resist_model(aerial, a, t)
#         cost = compute_cost(resist, target)
#         cost_history.append(cost)

#         # Gradients
#         resist_grad = a * resist * (1 - resist)
#         error_grad = 2 * (resist - target) * resist_grad
#         grad = gaussian_filter(error_grad, sigma=sigma)

#         # Regularization for binarization
#         bin_penalty = beta * (2 * mask * (1 - mask) * (1 - 2 * mask))
#         grad += bin_penalty

#         # Update
#         mask -= lr * grad
#         mask = np.clip(mask, 0, 1)

#         if i % 20 == 0 or i == iterations - 1:
#             print(f"Iteration {i+1}/{iterations}, Cost: {cost:.6f}")

#     return mask, cost_history

# # === 5. Option to Load External Image or Generate Pattern ===
# def load_target(shape, image_path=None):
#     Nx, Ny = shape

#     if image_path:
#         img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
#         if img is None:
#             raise ValueError(f"Could not load image from path: {image_path}")
#         resized_img = cv2.resize(img, (Ny, Nx))
#         normalized = resized_img / 255.0  # normalize to [0, 1]
#         return normalized
#     else:
#         # Fallback: Generate synthetic 3-bar pattern
#         X, Y = np.meshgrid(np.linspace(-1500, 1500, Nx), np.linspace(-1500, 1500, Ny))
#         target = np.zeros_like(X)
#         for x0 in [-100, 0, 100]:
#             target[(np.abs(X - x0) < 25) & (np.abs(Y) < 750)] = 1
#         return target

# # === 6. Run ILT Optimization ===
# def run_ilt(image_path=None):
#     Nx, Ny = 128, 128
#     sigma = 2.0
#     a = 10
#     t = 0.5
#     lr = 0.5
#     beta = 0.01
#     iterations = 200

#     # Load target pattern
#     target = load_target((Nx, Ny), image_path)
```